

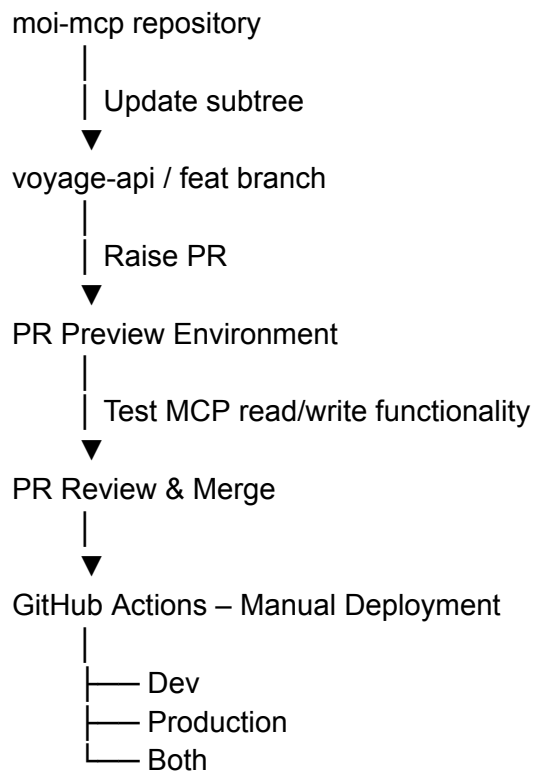
MOI MCP Gateway – Development, Preview & Deployment Runbook

1. Overview

The `moi-mcp` project is integrated into the `voyage-api` repository as a Git subtree under:

```
voyage-api/  
└─ moi-mcp/
```

The deployment flow is:



The deployment workflow is **manual**. Changes pushed or merged do not automatically deploy to Dev or Production.

2. Repository Structure

The standalone MCP repository is:

<https://github.com/sarvalabs/moi-mcp>

It is integrated into:

<https://github.com/sarvalabs/voyage-api>

Inside `voyage-api`:

```
voyage-api/  
├── moi-mcp/  
│   ├── src/  
│   ├── dist/  
│   ├── package.json  
│   └── ...  
├── .github/  
│   └── workflows/  
│       ├── moi-mcp.deploy.yml  
│       └── moi-mcp.preview.yml
```

The `moi-mcp` directory is maintained using **Git subtree**.

3. When the `moi-mcp` Repository Has New Changes

Whenever changes are made to the standalone `moi-mcp` repository and those changes need to be deployed through `voyage-api`, the latest code must first be pulled into the `voyage-api` repository.

Do **not** directly modify the subtree on the main deployment branch.

Create a branch first.

4. Create a Branch in voyage-api

Clone or enter the voyage-api repository:

```
cd voyage-api
```

Check the current branch and working tree:

```
git status
```

Create a new branch:

```
git checkout -b feat/update-moi-mcp
```

Use an appropriate branch name based on the change, for example:

```
feat/update-moi-mcp  
fix/moi-mcp-asset-creation  
chore/update-moi-mcp
```

5. Update the moi-mcp Subtree

If the upstream remote has not already been added, add it:

```
git remote add moi-mcp-upstream https://github.com/sarvalabs/moi-mcp.git
```

Fetch the latest changes:

```
git fetch moi-mcp-upstream
```

Pull the latest master branch into the moi-mcp subtree:

```
git subtree pull --prefix=moi-mcp moi-mcp-upstream master
```

This updates:

voyage-api/moi-mcp/

with the latest code from:

sarvalabs/moi-mcp

6. Verify the Updated Code

Check what changed:

`git status`

Review the changes:

`git diff`

You can also check the subtree history:

`git log --oneline -- moi-mcp`

Move into the MCP project:

`cd moi-mcp`

Install dependencies:

`npm ci`

Build the project:

`npm run build`

Make sure the build completes successfully before creating the PR.

Return to the repository root:

```
cd ..
```

7. Commit the Subtree Update

Review the files one more time:

```
git status
```

Stage the changes:

```
git add moi-mcp
```

Commit:

```
git commit -m "chore: update moi-mcp subtree"
```

Push the branch:

```
git push -u origin feat/update-moi-mcp
```

8. Create a Pull Request

Create a PR from the feature branch into the appropriate target branch.

Example:

```
feat/update-moi-mcp  
↓  
feat/moi-mcp
```

The PR should contain:

- What changed in `moi-mcp`
 - Why the change was made
 - Build/test results
 - Any MCP functionality that needs to be tested
 - Any configuration or environment changes, if applicable
-

9. PR Preview Environment

Opening or updating the PR triggers the MCP preview workflow:

`.github/workflows/moi-mcp.preview.yml`

The preview environment creates an isolated MCP instance for that PR.

For example, for PR 308:

Write:

`https://pr-308-mcp.preview.voyage.moi.technology`

Read:

`https://pr-308-mcp-read.preview.voyage.moi.technology`

The preview environment uses separate ports and PM2 processes for each PR.

Example:

PR 308

Read → 10616

Write → 10617

This allows the PR changes to be tested without changing the main Dev MCP deployment.

10. Test the PR Preview

After the preview deployment completes, verify the health endpoints.

Read:

```
curl -i https://pr-<PR_NUMBER>-mcp-read.preview.voyage.moi.technology/health
```

Write:

```
curl -i https://pr-<PR_NUMBER>-mcp.preview.voyage.moi.technology/health
```

Both should return HTTP 200.

The Read service should report:

```
{
  "ok": true,
  "network": "voyage",
  "readOnly": true
}
```

The Write service should report:

```
{
  "ok": true,
  "network": "voyage",
  "readOnly": false,
  "pairingMounted": true
}
```

Then connect the preview MCP endpoint to the MCP client/Claude and test the functionality introduced by the PR.

11. PR Review and Merge

Once the preview environment has been tested successfully:

1. Complete the required code review.
2. Verify the CI checks are successful.
3. Test the required MCP functionality.
4. Merge the PR into the MCP integration branch.

After merge, the latest `moi-mcp` code becomes part of the `voyage-api` deployment branch.

12. Manual Deployment to Dev / Production

Deployment is handled by:

`.github/workflows/moi-mcp.deploy.yml`

The workflow uses GitHub Actions `workflow_dispatch`.

There are three deployment options:

dev
production
both

Dev

Selecting:

dev

deploys only to the Dev MCP environment.

Production

Selecting:

production

deploys only to Production.

Both

Selecting:

both

deploys to both Dev and Production.

There is no automatic deployment on push or PR merge.

13. Trigger a Dev Deployment

Open:

GitHub → voyage-api → Actions

Select:

Deploy MOI MCP Gateway

Click:

Run workflow

Select:

deploy_target = dev

Run the workflow.

The workflow will:

Build



Deploy to Dev



Restart MOI MCP Read



Restart MOI MCP Write



Run Dev health checks

14. Trigger a Production Deployment

Only trigger Production after the changes have been validated in Dev.

Go to:

GitHub → voyage-api → Actions
→ Deploy MOI MCP Gateway
→ Run workflow

Select:

deploy_target = production

Run the workflow.

The workflow will:

Build

↓

Deploy to Production

↓

Restart MOI MCP Read

↓

Restart MOI MCP Write

↓

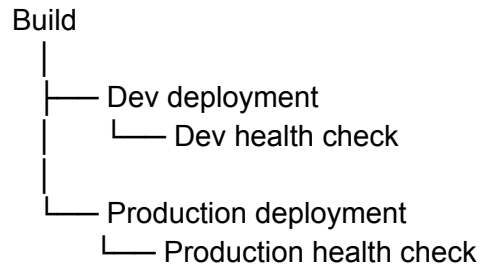
Run Production health checks

15. Deploy to Both Environments

If the same build needs to be deployed to both environments:

deploy_target = both

The workflow will execute:



16. MCP Deployment URLs

Dev

Write MCP

<https://dev-mcp.voyage.moi.technology>

Read MCP

<https://dev-mcp-read.voyage.moi.technology>

Production

Write MCP

<https://mcp.voyage.moi.technology>

Read MCP

<https://mcp-read.voyage.moi.technology>

The MCP endpoint is:

`/mcp`

For example:

<https://mcp.voyage.moi.technology/mcp>

17. Deployment Architecture

There are two MCP services:

Read Service

dist/http.js

Default port:

8787

Purpose:

Read-only MCP operations



Voyage RPC

Write Service

dist/server.js

Default port:

8788

Purpose:

Write MCP operations



WalletConnect / Reown approval



Voyage network

The Write MCP server does not hold the user's private key.

18. Dev Environment

Dev VM:

voyagedev

Application directory:

/opt/moi-mcp

Persistent data directory:

/var/lib/moi-mcp

PM2 processes:

moi-mcp-read
moi-mcp-write

Ports:

8787 → Read
8788 → Write

19. Production Environment

Production VM:

prod-voyage

Application directory:

/opt/moi-mcp

Persistent data directory:

/var/lib/moi-mcp

PM2 processes:

moi-mcp-read
moi-mcp-write

Ports:

8787 → Read
8788 → Write

20. Important Deployment Safety

The MCP deployment workflow only manages these two PM2 processes:

moi-mcp-read
moi-mcp-write

The deployment does not intentionally restart or delete unrelated applications or Docker services.

Before production deployment, always verify:

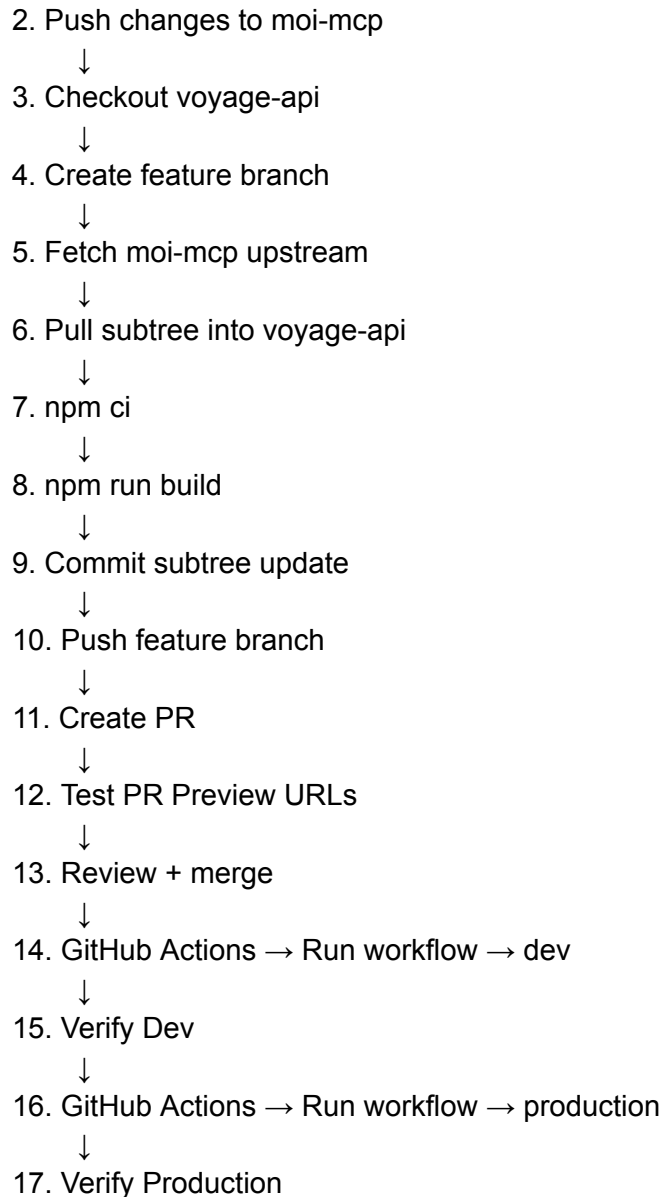
- PR has been reviewed.
 - Preview environment has been tested.
 - Dev deployment is healthy.
 - MCP functionality works as expected.
 - Correct production deployment option is selected.
-

21. Quick Operational Flow

For a normal moi-mcp update, follow this sequence:

1. Change code in moi-mcp repository

↓



23. Important Notes

- moi-mcp is maintained in its own repository and integrated into voyage-api using Git subtree.
- Always update the subtree from the upstream repository when bringing new moi-mcp changes into voyage-api.
- Test changes through the PR preview environment before merging.
- Dev and Production deployment are manually triggered through GitHub Actions.

- Selecting dev only affects Dev.
- Selecting production only affects Production.
- Selecting both affects both environments.
- Production should only be triggered after the changes have been validated.